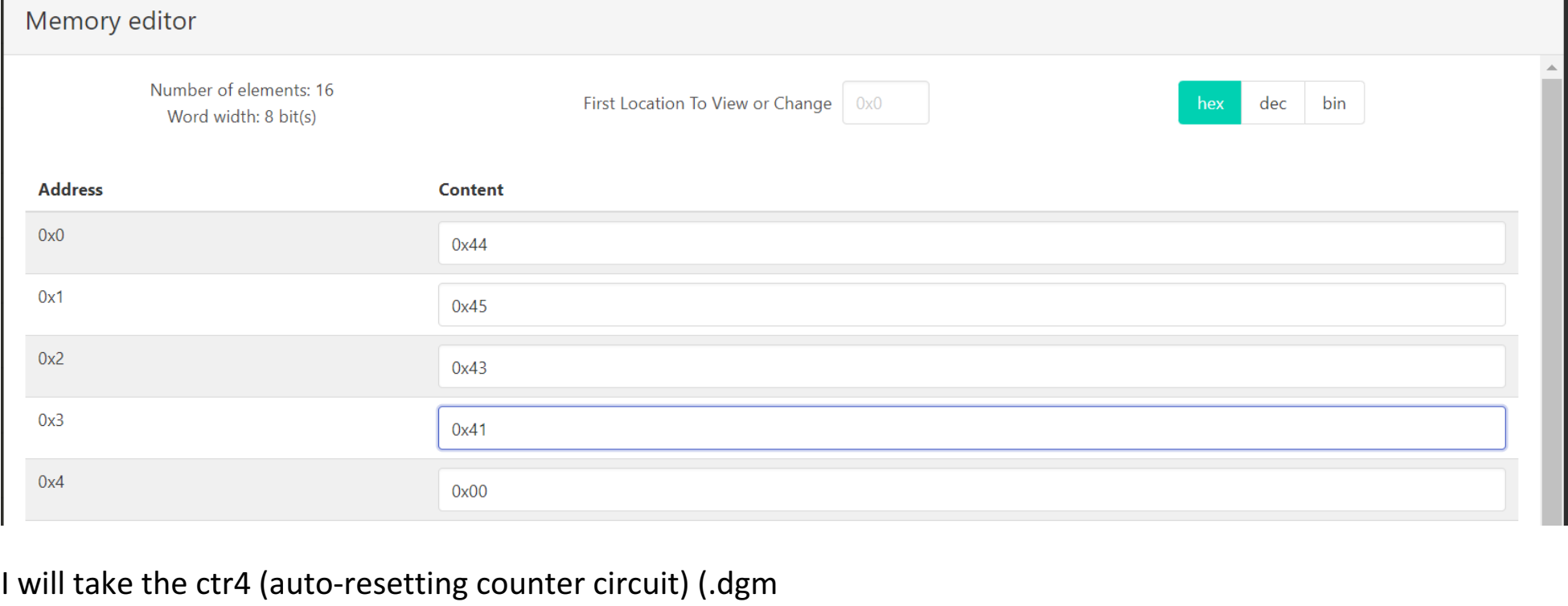


DECA 2.1 - Memory

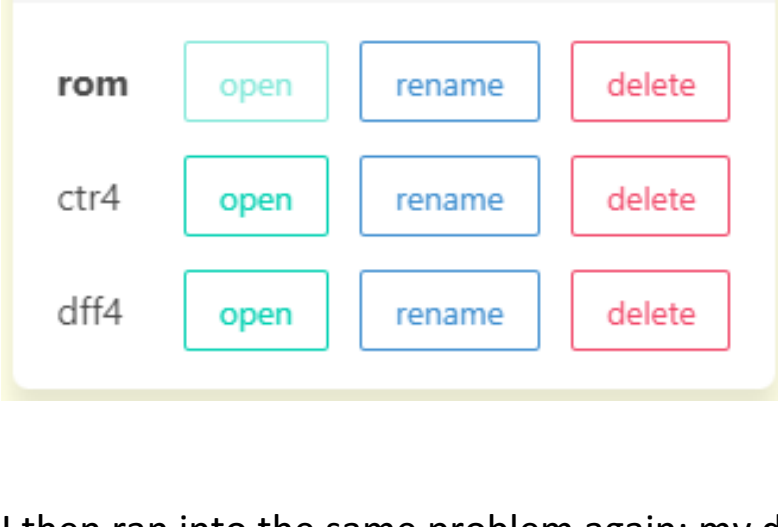
Sunday, December 11, 2022 4:02 PM

Part 1: Initialising ROM



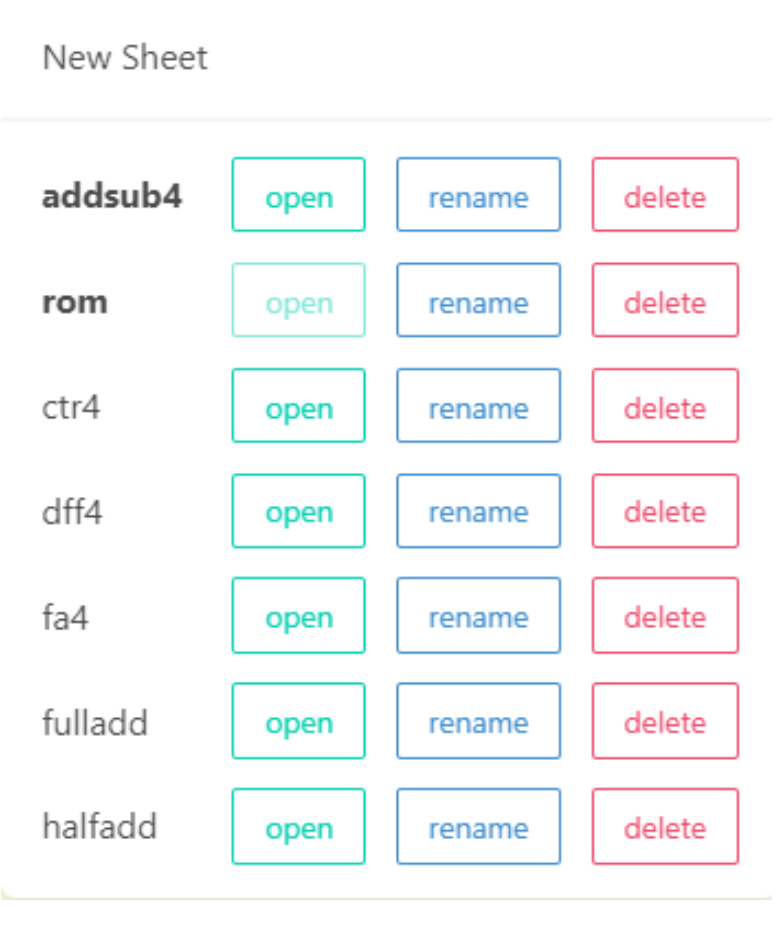
I will take the ctr4 (auto-resetting counter circuit) (.dgm file) from the Challenge for Part 1 Section 3. Setting the input to 4, I will have a counter that starts from 4 and counts down to 0, then resetting back to 4 and continuing. Theoretically, this should ensure that our DOUT port on the ROM outputs D, E, C, A repeatedly.

This did not work - Issie would not run the simulation. Update: I figured out that the problem had to do with the fact that my ctr4 circuit depends on my dff4 circuit (hierarchical design) which I had not copied into the Rom project. The solution should be straightforward, copying dff4.dgm into ROM.



I then ran into the same problem again: my dff4 block depends on my full adder block, which I have not imported into this project. I will fix this the same way, and also import my other basic blocks so as to not run into this again.

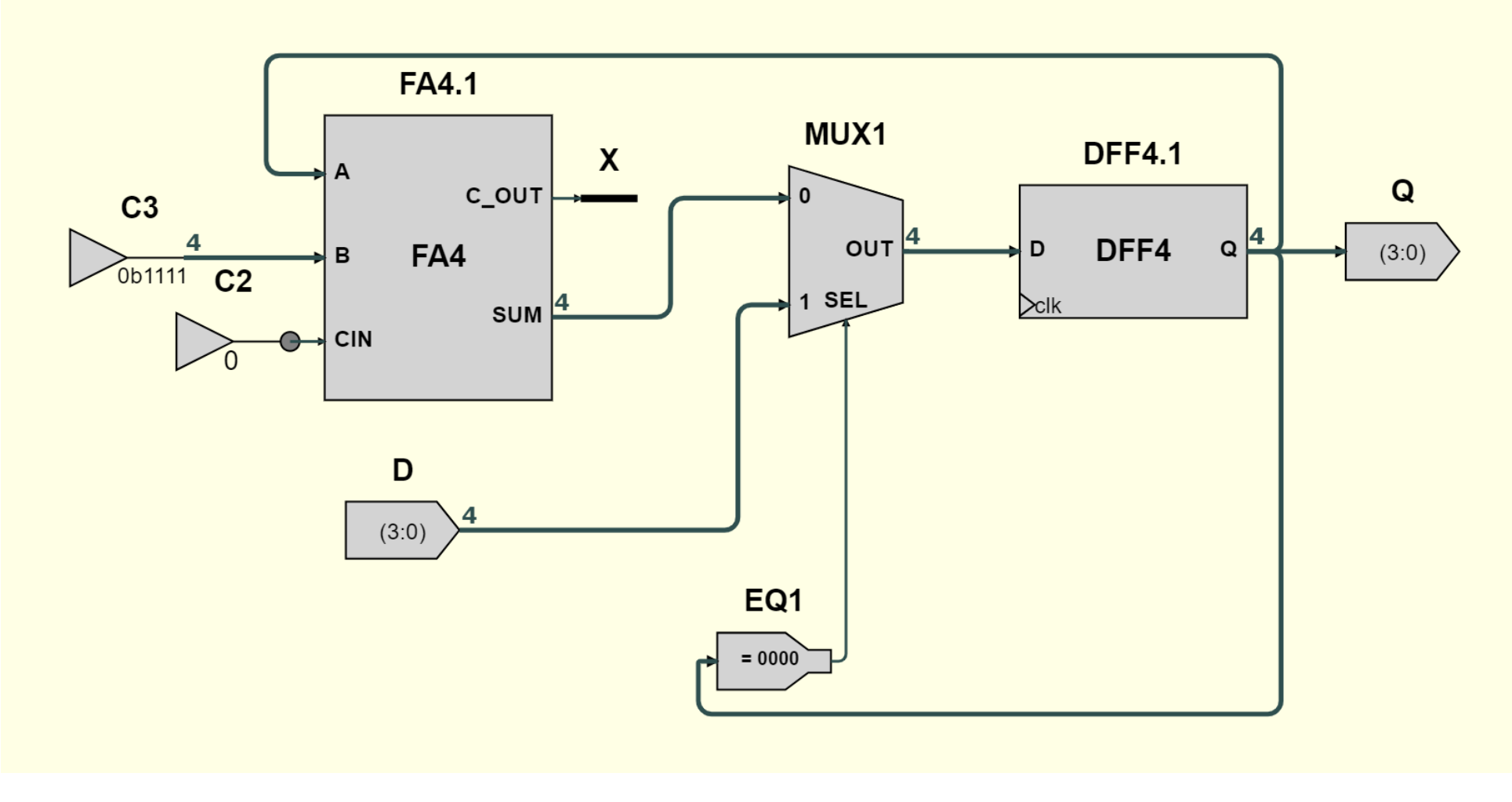
This is an excellent demonstration of hierarchical design.



I now have my basic arithmetic building blocks in place. The simulation now runs. The output of the ROM changes at each rising clock edge, looping through the values 0x44, 0x45, 0x43 and 0x41.

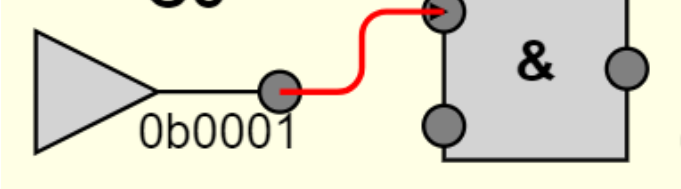
Part 2: Challenge - Message writer

I realised only after reading the Challenge that I had gone further than I needed to for the first part of this section - I had already had the idea to implement a counter circuit as the ADDR input for the ROM. However, as per the specifications listed in the instructions, my pre-built ctr4 circuit will not be adequate for this task.



Instead of the counter always counting down from 0, we need it to only increment when a LOAD input is set to 1. Additionally, we want to be counting up, not down. This is an easy change: we can change the constant C3 to 0001 instead of 1111. And in order to implement the LOAD part, I will use an AND gate with that constant.

However, simply connecting an AND gate does not work, because of the different bus widths:

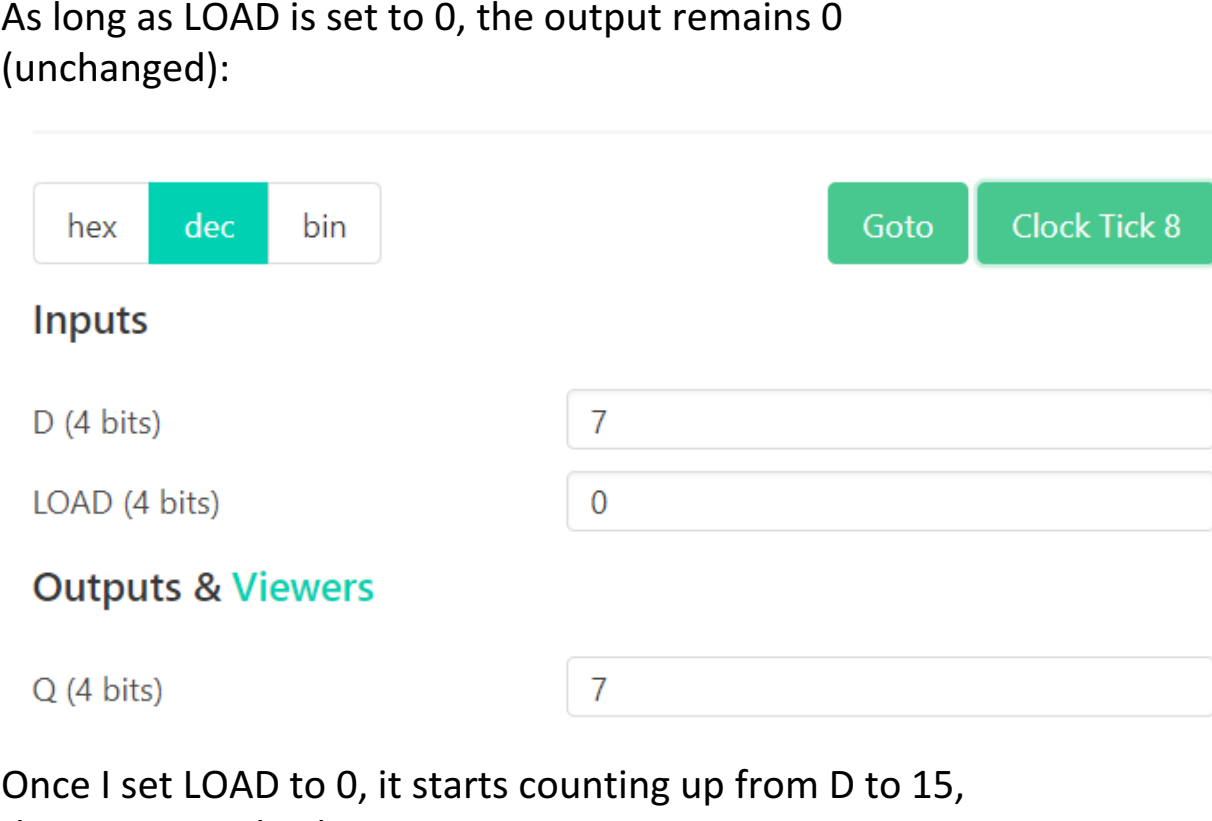


So I will use a Bus Select to compare the least significant bit of C3 to the LOAD input.

While writing the previous sentence, I realised I could make my circuit much simpler by simply setting LOAD to be the constant. This way the adder will add 1 when LOAD is 1, and add 0 (no change) when LOAD is 0. I will still however have to deal with the bus width issue.

Let us test the functionality of this counter circuit before we go any further.

As long as LOAD is set to 0, the output remains 0 (unchanged):

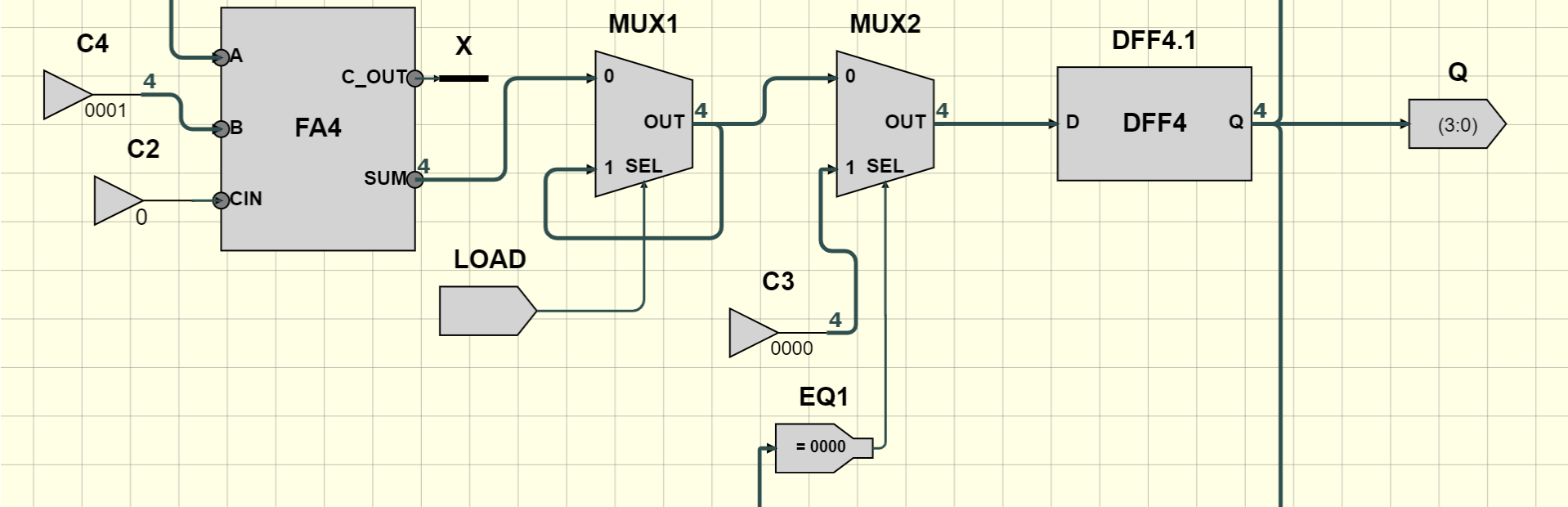


Once I set LOAD to 0, it starts counting up from D to 15, then resetting back to D.

This is not the behaviour we want.

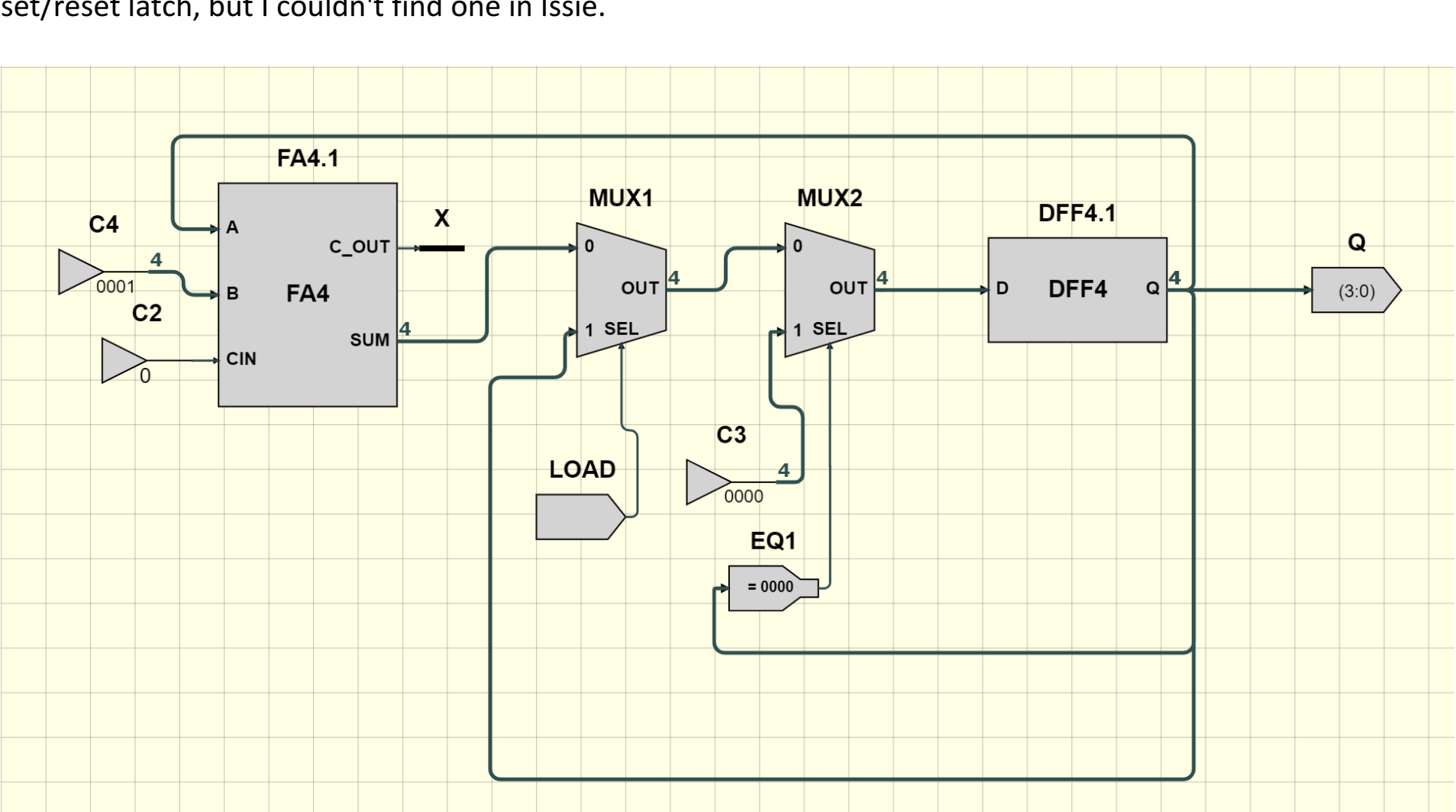
We have $D \leq \text{ctr} < 16$, but we want $0 \leq \text{ctr} < D$.

Update:



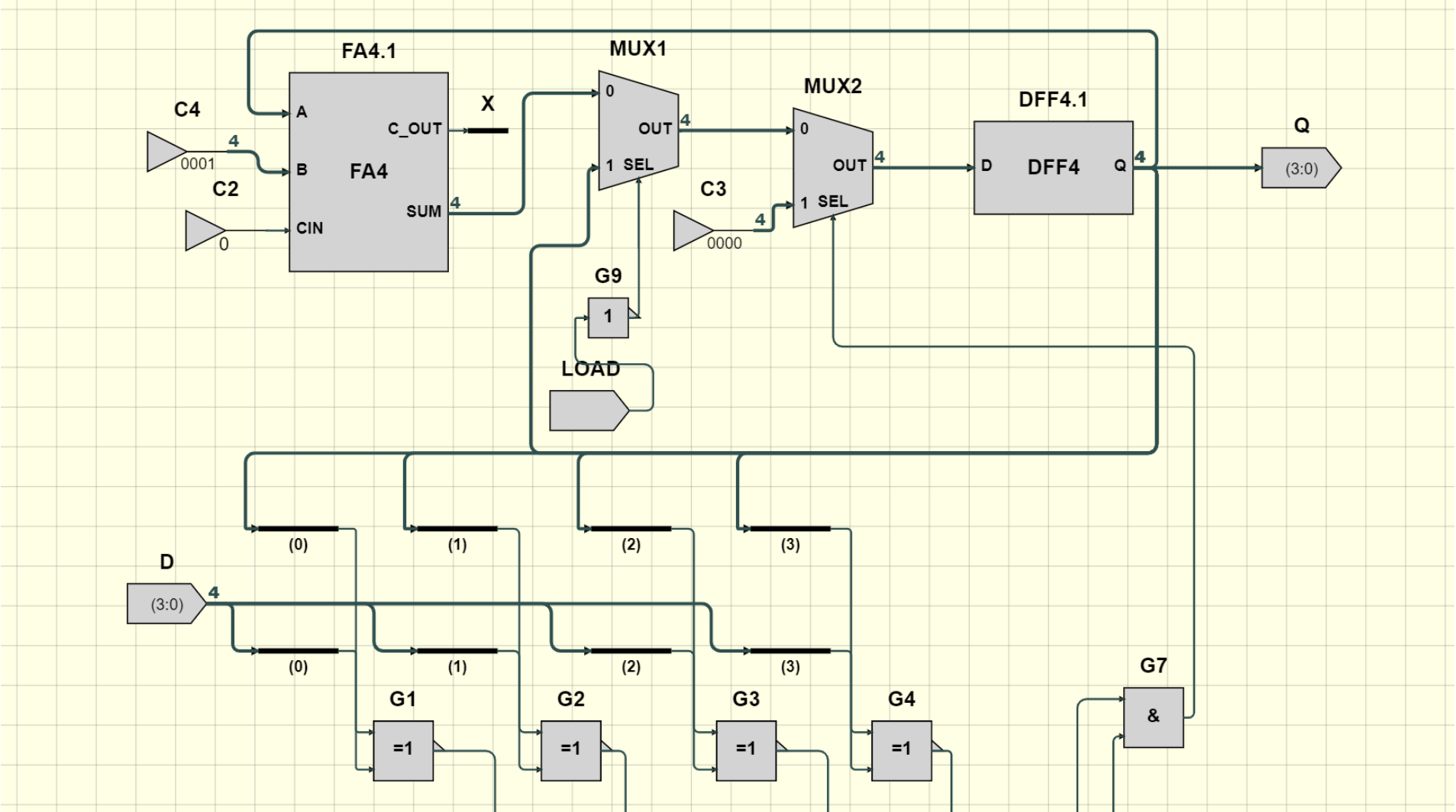
I tried this solution, but since the output of the MUX feeds back into itself, Issie will not run the simulation.

I believe I could fix this problem with a flip flop with a set/reset latch, but I couldn't find one in Issie.



I believe this implementation solves my problem, however it is missing the value D.

I want the Select line of MUX2 to be 1 if $Q = D$, and to be 0 otherwise.



In order to implement that, I had to make a bitwise comparator.

Here is the final counter circuit, and after testing, it works correctly.

We can now implement it into the original circuit, and conclude the challenge:

